INFO 101 – Introduction to Computing and Security / Assignment 4

Prof. Dr. Rui Abreu
University of Porto, Portugal
rui@computer.org

December 3, 2022

1 Introduction

The following exercises are for the INFO 101 course in the year 2021. Solutions for the exercises have to be provided via Moodle until December 11.

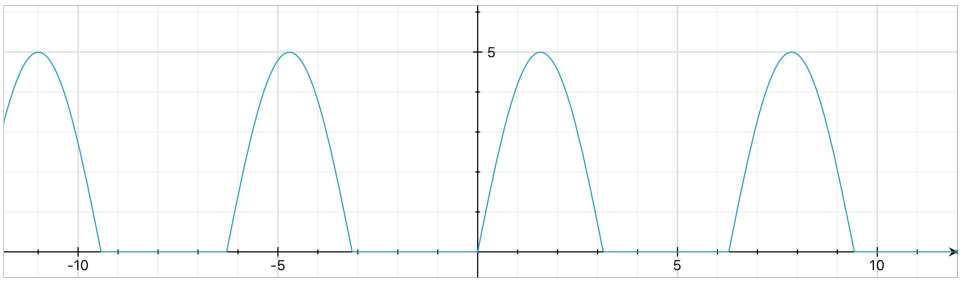
The solutions have to be provided in one pdf file, with the (English) name and student identification number. **Note that given the deadlines are strict!** During the practical part students are asked to explain their solution at the black board. In case a provided solution cannot be sufficiently explained, I will evaluate this part of the practical exercise as not provided!

2 LISP Programming

Exercise 1: Write a function in LISP using `defun` that returns the value of the logical bijunction $\leftrightarrow$.

Exercise 2: Write a function in LISP using `defun` that takes a list as input and returns another list, where the last element is the first of the given list. The other elements are not changed.

Exercise 3: Write a function in LISP that returns the following mathematical function. Use `if` to distinguish the cases. Note that the function is based on $\sin()$.



Exercise 4: Write a LISP function `gen-list` that takes a non-negative integer n as input, and returns a list that comprises n empty sets. For example:

`(gen-list 0)` returns `NIL`
`(gen-list 1)` returns `(NIL`

`(gen-list 2)` returns `(NIL NIL`
`(gen-list 5)` returns `(NIL NIL NIL NIL NIL)`

Exercise 5: Write a recursive LISP function that implements the Fibonacci numbers defined as follows:

$$F(x) = \begin{cases} 0 & \text{if } x = 0 \\ 1 & \text{if } x = 1 \\ F(x-1) + F(x-2) & \text{otherwise} \end{cases}$$

Exercise 6: Write a LISP function `my-member` that returns `T` is the given argument is element of the given list, and `nil` otherwise. Here are some examples:

`(my-member 'a '())` returns `nil`
`(my-member 'a '(c b a))` returns `T`
`(my-member 'a '(c b (a)))` returns `nil` (because `a` is not element of the list but a list that contains `a`.

Exercise 7: This is an extension to **Exercise 6**. Write a new LISP function `deep-member` that returns `T` if the given argument is element of the given list or in any of its sublists, and `nil` otherwise. Here are some examples:

`(deep-member 'a '())` returns `nil`
`(deep-member 'a '(c b a))` returns `T`
`(deep-member 'a '(c b (a)))` returns `T`
`(deep-member 'a '(c b (1 (a))))` returns `T`
`(deep-member 'a '(c b (1 (d))))` returns `nil`

Exercise 8: Write a LISP function `binary-to-int` that converts a binary number that is given as list comprising only `0` and `1` into a integer. For example:

`(binary-to-int '())` returns `nil`, because the empty set is not a representation of a binary number
`(binary-to-int '(1 0))` returns `2`
`(binary-to-int '(0 1))` returns `1`
`(binary-to-int '(0))` returns `0`
`(binary-to-int '(1 1 1 0))` returns `14`

Exercise 9: Write a LISP function `int-to-binary` that converts a given non-negative integers into a list representing binary numbers using `0` and `1`. For example:

`(int-to-binary 0)` returns `(0)`
`(int-to-binary 1)` returns `(1)`
`(int-to-binary 2)` returns `(1 0)`
`(int-to-binary 13)` returns `(1 1 0 1)`

Note: You may consider using the LISP functions `floor` *together with* `/`*, and* `rem` *to solve this example.*

Exercise 10: Write a LISP function `add-elements` that takes a symbol s as first argument, a positive integer n as second argument, and a list of length $\leq n$ as third argument. `add-elements` returns a new list, that comprises n minus the length of the list elements s first and the original list afterwards. For example:

`(add-elements 'x 2 '(1 0))` returns `(1 0)`
`(add-elements 'x 4 '(1 0))` returns `(x x 1 0)`
`(add-elements 'x 4 '(a))` returns `(x x x a)`